

```

%Joseph Dearing
function tpair = pairtime(posa,vela,posb,velb,twosigma)
%taking inputs as 1x2 row vectors for postion and velocity
%need to input 2*sigma as sigma=r and collision occurs when positions are
%2*r seperated
del_x=[posa(1)-posb(1),posa(2)-posb(2)];
del_x_sq=del_x(1)^2+del_x(2)^2;
del_v=[vela(1)-velb(1),vela(2)-velb(2)];
del_v_sq=del_v(1)^2+del_v(2)^2;
scal=del_x(1)*del_v(1)+del_x(2)*del_v(2);
upsilon=scal^2-del_v_sq*(del_x_sq-4*twosigma^2);
if upsilon>0
    if scal<0
        tpair=-(scal+sqrt(upsilon))/del_v_sq;
    else
        tpair=inf;
    end
else
    tpair=inf;
end
end

```

```

function twall = walltime(posa,vela,sigma)
%all inputs taken as integers, hence in eventdisks we need to consider x
%and y wall collisions
if vela>0
    twall=(1-posa-sigma)/vela;
elseif vela<0
    twall=(posa-sigma)/abs(vela);
else
    twall = inf;
end
end

```

```

function eventdisks(pos,vel,sigma,N,n_events)
t=0;
walltimes=zeros(1,2*N);
pairs=nchoosek(1:N,2);%create all possible pairs for N disks
pairtimes=zeros(1,nchoosek(N,2));
t_sample=0;
dt=0.1;%seperation of time for which we take samples
%optimising code with preallocation as it was running slowly
est_length=n_events*20*N;
x_history=zeros(est_length,1);
c=1;
for i = 1:n_events
    %computing all wall collision times
    for j =1:N
        walltimes(j)=walltime(pos(j,1),vel(j,1),sigma/2);
        walltimes(j+N)=walltime(pos(j,2),vel(j,2),sigma/2);
    end
    %computing all pair collision times

```

```

for k=1:nchoosek(N,2)
    pairtimes(k)=pairtime(pos(pairs(k,1,:),:),vel(pairs(k,1,:),:), ...
        pos(pairs(k,2,:),:),vel(pairs(k,2,:),:),sigma);
end
%determining event at with least time to occur
[m_w,w_i]=min(walltimes);
[m_p,p_i]=min(pairtimes);
if m_w<m_p
    m=m_w;
    event_type = 'wall';
    mindex = w_i;
else
    m=m_p;
    event_type = 'pair';
    mindex = p_i;
end
%recording values of x up until time of collision
while t_sample < (t + m)
    pos_at_sample = pos + vel * (t_sample - t);
    %more optimisation
    if c+N > length(x_history)
        x_history(end + 100000) = 0;
    end
    x_history(c:c+N-1)=pos_at_sample(:,1);
    c=c+N;
    t_sample=t_sample+dt;
end
%updating time and position
t=t+m;
pos=pos+vel*m;
%updating velocity dependant on which collision occurs
if strcmp(event_type,'wall')
    %collision with wall x or y reverses direction of velocity of
disk
    %in x or y
    if mindex<=N
        vel(mindex,1)=-1*vel(mindex,1);
    else
        vel(mindex-N,2)=-1*vel(mindex-N,2);
    end
else
    pair=pairs(mindex,:);
    del_x=[pos(pair(1),1)-pos(pair(2),1),pos(pair(1),2)-
pos(pair(2),2)];
    abs_x=sqrt(del_x(1)^2+del_x(2)^2);
    e_perp=del_x/abs_x;
    del_v=[vel(pair(1),1)-vel(pair(2),1),vel(pair(1),2)-
vel(pair(2),2)];
    vel(pair(1,:),:)=vel(pair(1,:),:)-e_perp*(dot(del_v,e_perp));
    vel(pair(2,:),:)=vel(pair(2,:),:)+e_perp*(dot(del_v,e_perp));
end
end
x_history = x_history(1:c-1);

```

```

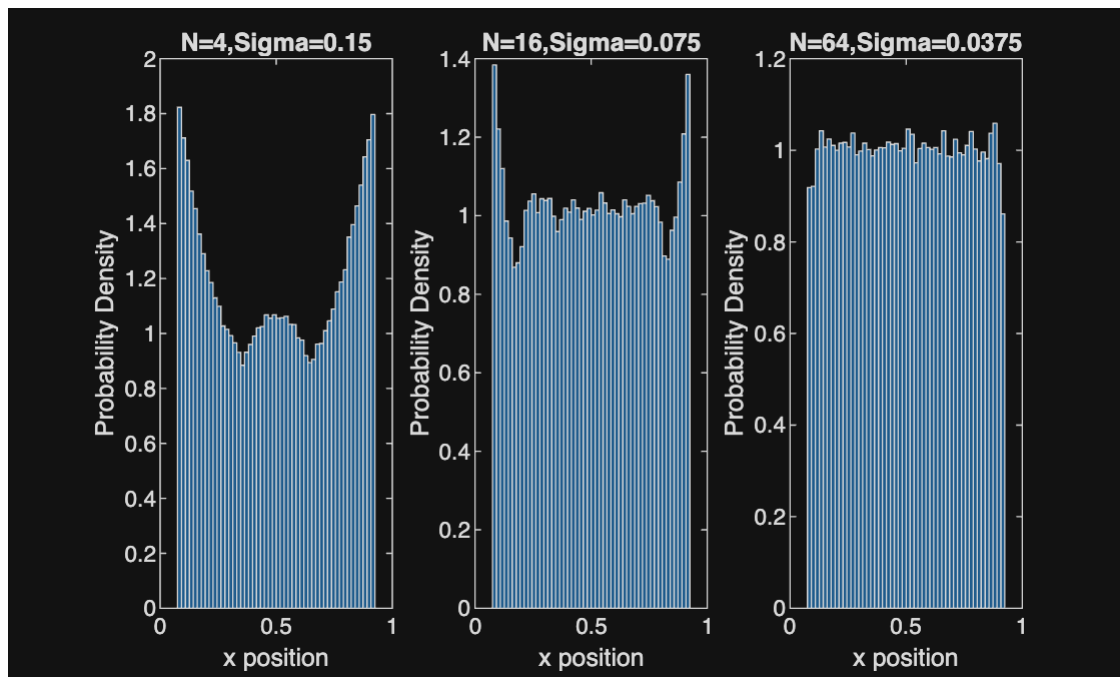
histogram(x_history,"BinLimits",
[0.075,0.925],"NumBins",50,'Normalization','pdf');
title(['N=',num2str(N),'Sigma=',num2str(sigma)]);
xlabel('x position');
ylabel('Probability Density');
xlim([0, 1]);
end

```

```

Nlist=[4,16,64];
sigma_og=0.15;
N_og=4;
n_events=100000;
figure;
for i=1:length(Nlist)
    N=Nlist(i);
    %scaling sigma so density remains the same
    %N1*pi*sigma1^2=N2*pi*sigma2^2
    sigma=sigma_og*sqrt(N_og/N);
    %spacing disks evenly across square at t=0
    side=ceil(sqrt(N));
    spacing=1/(side+1);
    [X,Y]=meshgrid(spacing:spacing:1-spacing);
    pos=[X(:),Y(:)];
    pos=pos(1:N,:);
    %choosing velocity to be rand(-1,1)
    vel=2*rand(N,2)-1;
    subplot(1,3,i);
    eventdisks(pos,vel,sigma,N,n_events);
end

```



We can see that for N=4 we recover the "Batman" density. Furthermore, by increasing N we can see that the features of the "Batman" flatten out. When N=4 due to the size of the disks, they struggle to form a uniform

"fluid" in the middle. However, after increasing N sufficiently the phenomenon becomes less prevalent and the disks form a more uniform "fluid".

```
function eventdiskvel(pos,vel,sigma,N,n_events)
%modified code to sample velocity, and plot such with gaussian overlay
t=0;
walltimes=zeros(1,2*N);
pairs=nchoosek(1:N,2);
pairtimes=zeros(1,nchoosek(N,2));
t_sample=0;
dt=0.1;
est_length=n_events*10;
vx_history=zeros(est_length,1);
c=1;
for i = 1:n_events
    for j = 1:N
        walltimes(j)=walltime(pos(j,1),vel(j,1),sigma/2);
        walltimes(j+N)=walltime(pos(j,2),vel(j,2),sigma/2);
    end
    for k=1:nchoosek(N,2)
        pairtimes(k)=pairtime(pos(pairs(k,1,:),:),vel(pairs(k,1,:),:), ...
            pos(pairs(k,2,:),:),vel(pairs(k,2,:),:),sigma);
    end
    [m_w,w_i]=min(walltimes);
    [m_p,p_i]=min(pairtimes);
    if m_w<m_p
        m=m_w;
        event_type = 'wall';
        mindex = w_i;
    else
        m=m_p;
        event_type = 'pair';
        mindex = p_i;
    end
    while t_sample<t+m
        if c>length(vx_history)
            vx_history(end+10000)=0;
        end
        vx_history(c)=vel(1,1);
        c=c+1;
        t_sample=t_sample+dt;
    end
    t=t+m;
    pos=pos+vel*m;
    if strcmp(event_type,'wall')
        if mindex<=N
            vel(mindex,1)=-1*vel(mindex,1);
        else
            vel(mindex-N,2)=-1*vel(mindex-N,2);
        end
    else
        pair=pairs(mindex,:);
```

```

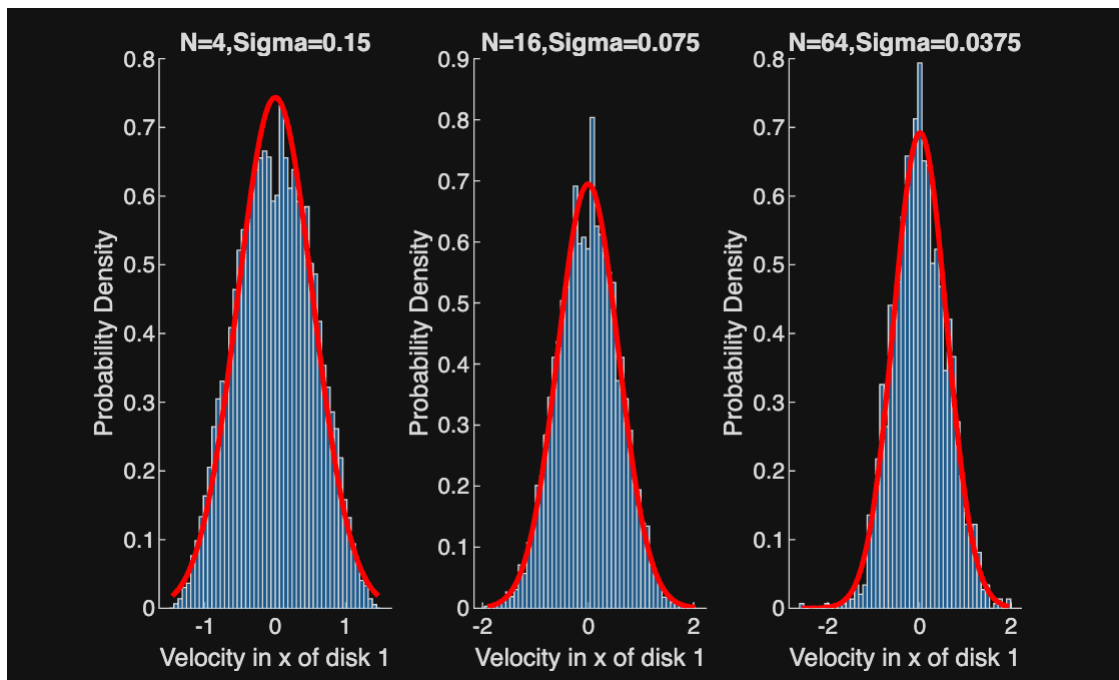
        del_x=[pos(pair(1),1)-pos(pair(2),1),pos(pair(1),2)-
pos(pair(2),2)];
        abs_x=sqrt(del_x(1)^2+del_x(2)^2);
        e_perp=del_x/abs_x;
        del_v=[vel(pair(1),1)-vel(pair(2),1),vel(pair(1),2)-
vel(pair(2),2)];
        vel(pair(1),:)=vel(pair(1),:)-e_perp*(dot(del_v,e_perp));
        vel(pair(2),:)=vel(pair(2),:)+e_perp*(dot(del_v,e_perp));
    end
end
vx_history=vx_history(1:c-1);
mu=mean(vx_history);
sd=std(vx_history);
x=linspace(min(vx_history),max(vx_history),100);
y=1/(sd*sqrt(2*pi))*exp(-0.5*((x-mu)/sd).^2);
hold on;
histogram(vx_history,50,'Normalization','pdf');
plot(x,y,'r','LineWidth',2)
title(['N=',num2str(N),'Sigma=',num2str(sigma)]);
xlabel('Velocity in x of disk 1');
ylabel('Probability Density');
hold off;
end

```

```

Nlist=[4,16,64];
sigma_og=0.15;
N_og=4;
n_events=100000;
figure;
for i=1:length(Nlist)
    N=Nlist(i);
    sigma=sigma_og*sqrt(N_og/N);
    side=ceil(sqrt(N));
    spacing=1/(side+1);
    [X,Y]=meshgrid(spacing:spacing:1-spacing);
    pos=[X(:),Y(:)];
    pos=pos(1:N,:);
    vel=2*rand(N,2)-1;
    subplot(1,3,i);
    eventdiskvel(pos,vel,sigma,N,n_events);
end

```



By initialising the velocity as $\text{rand}(-1,1)$ we have a variance of $1/3$. Meaning we have an average kinetic energy of $1/3$. The simulation conserves energy hence the initial variance must be equal to the final variance. Hence the parameter of the Gaussian is $\sigma = \sqrt{1/3}$

```

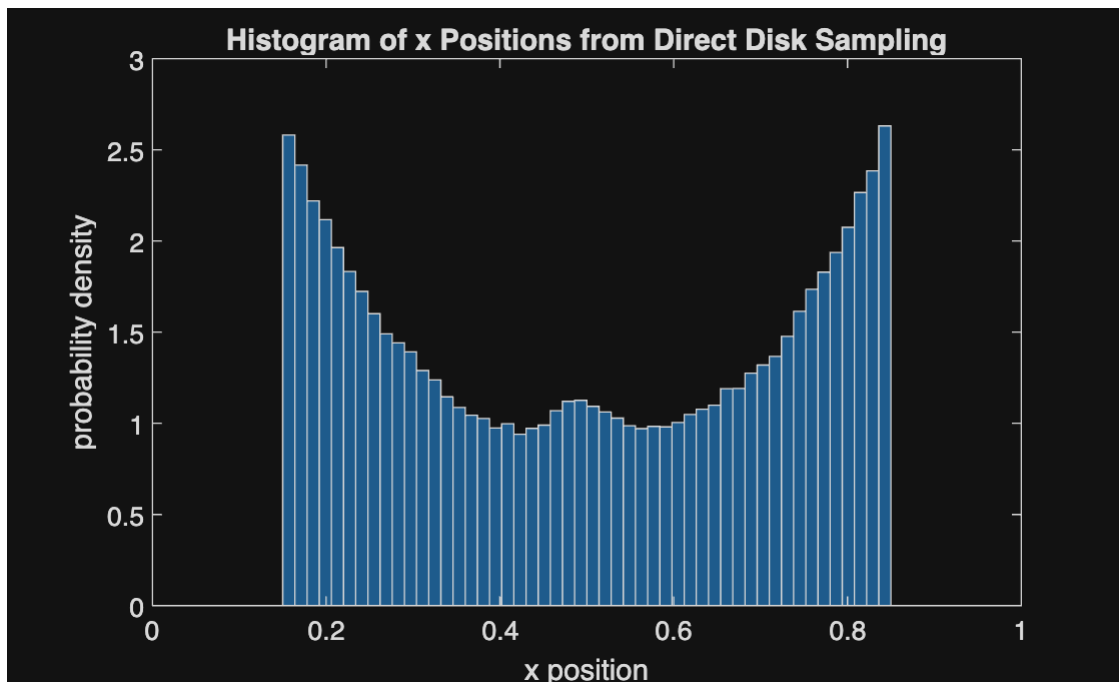
%Joseph Dearing
function samples = directdisk(n_samples,N,sigma)
samples=zeros(N,n_samples);
c=0;
while c<n_samples
    x=zeros(1,N);
    y=zeros(1,N);
    valid=true;
    %creating positions for each disk
    for i =1:N
        x(i)=sigma+(1-2*sigma)*rand();
        y(i)=sigma+(1-2*sigma)*rand();
        %determining distance between each pair of disks
        for j=1:i-1
            d=sqrt((x(i)-x(j))^2+(y(i)-y(j))^2);
            if d < 2 * sigma
                %if disks too close update valid
                valid = false;
                break;
            end
        end
        if ~valid
            %if not valid we restart loop
            break;
        end
    end
    if valid
        %if valid record and increase count
        c=c+1;
        samples(:,c)=x;
    end
end
samples=samples(:)';
end

```

```

xpos=directdisk(50000,4,0.15);
histogram(xpos,"Numbins",50,"Normalization","pdf")
xlim([0,1])
xlabel('x position')
ylabel('probability density')
title('Histogram of x Positions from Direct Disk Sampling');

```



We can see that the direct-disk algorithm gives the same batman figure. Both algorithms sample the distribution of the same system. Direct disks does so through independant samples whilst event disks does it through correlated samples. Both of which converge to the same stationary distribution. Any devitions should be due to noise or insufficient sampling.

```
function samples=markovdisks(pos,N,sigma,n_samples,delta)
samples=zeros(N,n_samples);
for i = 1:n_samples
    k=randi(N);
    dx=delta*(2*rand()-1);
    dy=delta*(2*rand()-1);
    xtrial=pos(1,k)+dx;
    ytrial=pos(2,k)+dy;
    valid=true;
    %checking if inside box
    if xtrial<sigma || xtrial>1-sigma || ytrial<sigma || ytrial>1-sigma
        valid=false;
    end
    %checking for disk overlap
    if valid
        for j=1:N
            if j==k
                continue
            end
            d=(xtrial-pos(1,j))^2+(ytrial-pos(2,j))^2;
            if d<4*(sigma)^2
                %checking if disks overlap
                valid=false;
                break;
            end
        end
    end
end
%unlike for direct, we record postion regardless
```



```

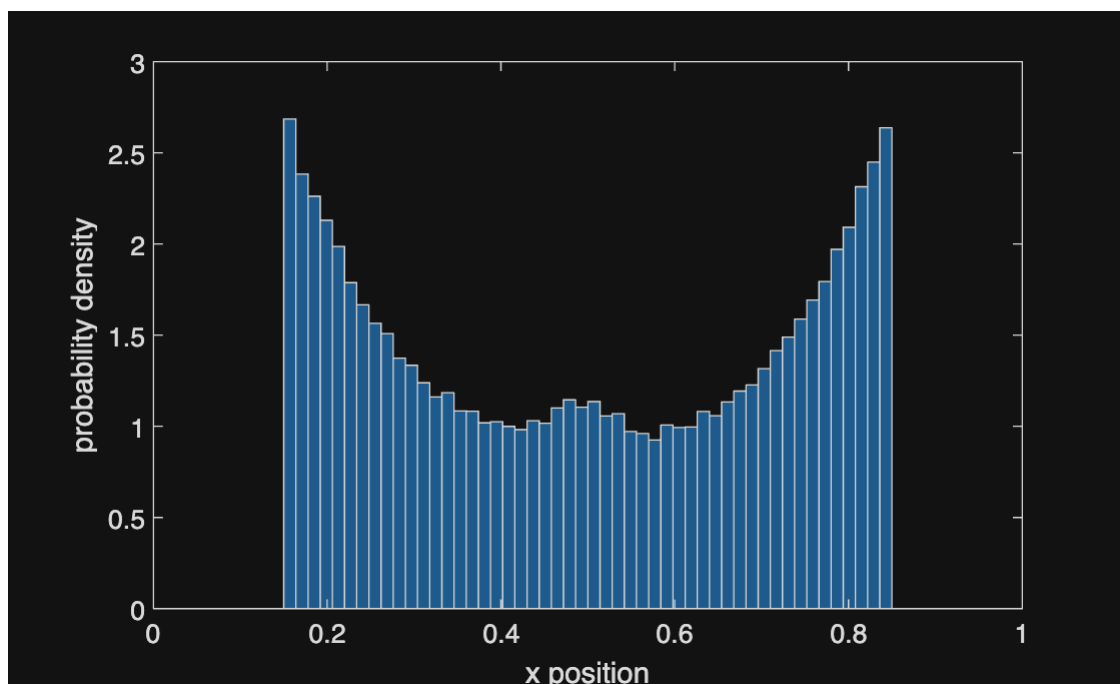
%update positions if no overlap then record
if valid
    pos(1,k)=xtrial;
    pos(2,k)=ytrial;
end
samples(:,i)=pos(1,:);
end
samples=samples(:)';
end

```

```

pos=[0.25,0.25,0.75,0.75;0.25,0.75,0.25,0.75];
xsamples=markovdisks(pos,4,0.15,500000,0.1);
histogram(xsamples,"NumBins",50,"Normalization","pdf")
xlabel('x position')
ylabel('probability density')
xlim([0,1]);

```



Again we see the batman figure. Furthermore, we again expect the output to be exactly the same up to some error or insufficient sampling due to the fact they sample the exact same distribution. Deviations will occur due to such and not the fact $N=4$.

```

%Joseph Dearing
function xsamples=levyharmonic(N,L,n_events)
xsamples=zeros(n_events,N+1);
%indexing changed to align with that of Matlab
for i = 1:n_events
    x=zeros(1,N+1);
    x(1)=L*rand();
    x(N+1)=x(1)+L;
    for k = 2:N
        xbar=((N-(k-1))*x(k-1)+x(N+1))/(N-(k-1)+1);
        sigma=sqrt(1/(1+1/(N-(k-1))));
        x(k)=xbar+sigma*randn();
    end
    xsamples(i,:)=x;
end
end

```

```

function xsamples=metroharmonic(pos,N,L,n_events,delta)
xsamples=zeros(n_events+1,N+1);
xsamples(1,:)=pos;
for i =1:n_events
    k=randi([1,N]);
    dx=delta*(2*rand()-1);
    kplus=mod(k,N)+1;
    kminus=mod(k-2,N)+1;
    xplus=xsamples(i,kplus);
    xminus=xsamples(i,kminus);
    if k==N
        xplus=xplus+L;
    end
    if k==1
        xminus=xminus-L;
    end
    xk=xsamples(i,k);
    xdash=xk+dx;
    U=1/2*((xplus-xk)^2+(xk-xminus)^2);
    Udash=1/2*((xplus-xdash)^2+(xdash-xminus)^2);
    upsilon=rand();
    xout=xsamples(i,:);
    if upsilon<exp(-(Udash-U))
        xout(k)=xdash;
    end
    xout(N+1)=xout(1)+L; %Enforcing periodicity
    xsamples(i+1,:)=xout;
end
end

```

```

function xsamples=hmcharmonic(pos,N,L,n_events,epsilon,I)
xsamples=zeros(n_events+1,N+1);
xsamples(1,:)=pos;
for i=1:n_events

```

```

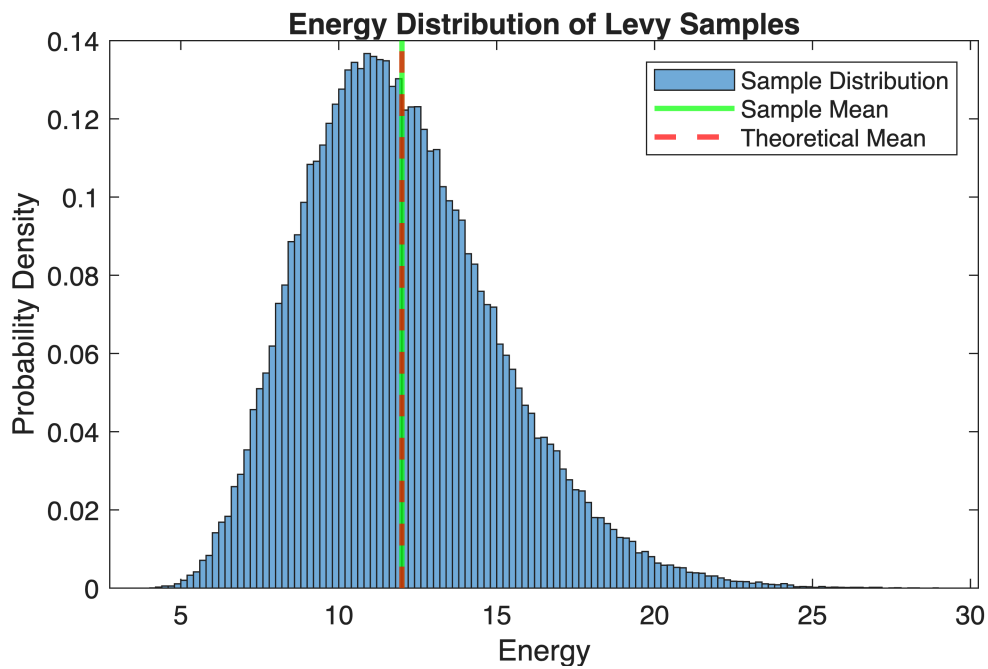
x=xsamples(i,:);
xtilda=x;
p=randn(1,N);
E=0.5*(sum(diff(x).^2))+0.5*sum(p.^2);
grad=2*xtilda(1:N)-xtilda(2:N+1)-[xtilda(N)-L,xtilda(1:N-1)];
p=p-0.5*epsilon*grad;
for l =1:I
    xtilda(1:N)=xtilda(1:N)+epsilon*p;
    xtilda(N+1)=xtilda(1)+L;
    if l<I
        grad=2*xtilda(1:N)-xtilda(2:N+1)-[xtilda(N)-L,xtilda(1:N-1)];
        p=p-0.5*epsilon*grad;
    end
end
grad=2*xtilda(1:N)-xtilda(2:N+1)-[xtilda(N)-L,xtilda(1:N-1)];
p=p-0.5*epsilon*grad;
Etilda=0.5*(sum(diff(xtilda).^2))+0.5*sum(p.^2);
upsilon=rand();
if upsilon<exp(-(Etilda-E))
    x=xtilda;
end
xsamples(i+1,:)=x;
end
end

```

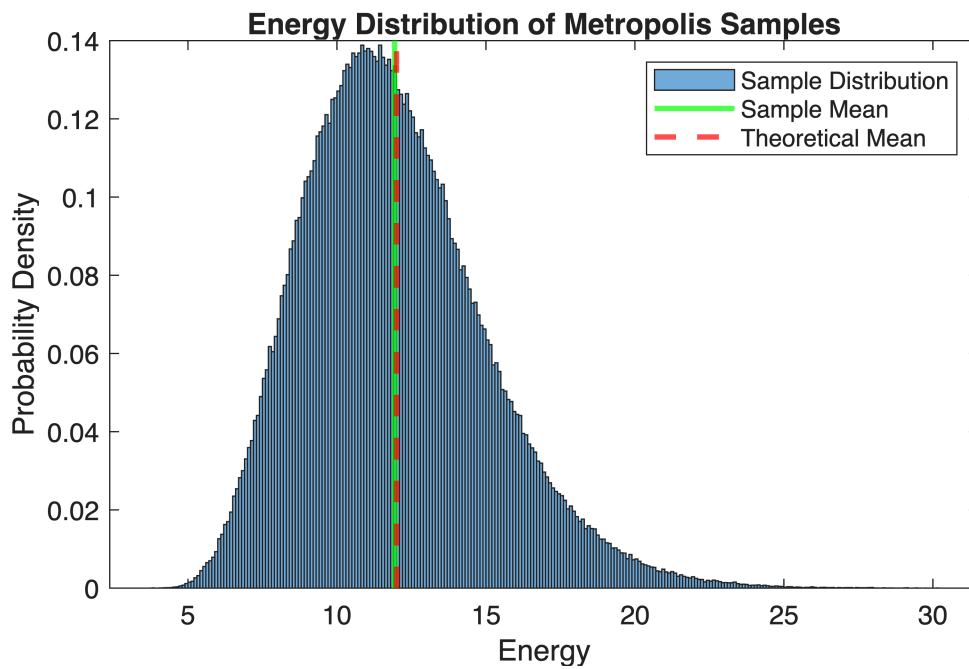
```

%Levy
N=20;
L=10;
n_events=100000;
xsamples=levyharmonic(N,L,n_events);
diffs=diff(xsamples,1,2);
%E=sum from k=1 to N of (x.k-x.(k-1))^2
E=0.5*sum(diffs.^2,2);
Emean_theoretical=(L^2/N+N-1)/2;
Emean_sample=mean(E);
histogram(E,"Normalization","pdf");
xline(Emean_sample,'g','LineWidth',2);
xline(Emean_theoretical,'--r','LineWidth',2);
xlabel("Energy");
ylabel("Probability Density");
title('Energy Distribution of Levy Samples');
legend('Sample Distribution','Sample Mean','Theoretical Mean');

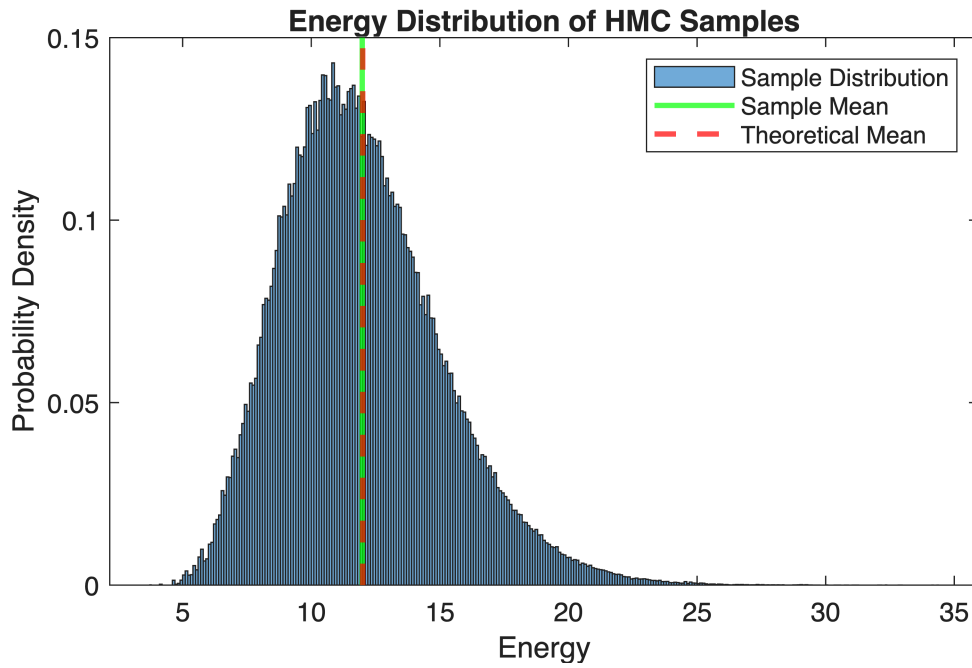
```



```
%Metro
N=20;
L=10;
n_events=1000000;
delta=1;
pos=(0:N)*L/N;
xsamples=metroharmonic(pos,N,L,n_events,delta);
diffs=diff(xsamples,1,2);
E=0.5*sum(diffs.^2,2);
burn=floor(0.1*n_events);%Getting rid of the initial 10% of data to
eliminate bias towards original position input.
E = E(burn+1:end);
Emean_theoretical=(L^2/N+N-1)/2;
Emean_sample=mean(E);
histogram(E,"Normalization","pdf");
xline(Emean_sample,'g','LineWidth',2);
xline(Emean_theoretical,'--r','LineWidth',2);
xlabel("Energy");
ylabel("Probability Density");
title('Energy Distribution of Metropolis Samples');
legend('Sample Distribution','Sample Mean','Theoretical Mean');
```



```
%HMC
N=20;
L=10;
n_events=1000000;
epsilon=0.1;
I=20;
pos=(0:N)*L/N;
xsamples=hmcharmonic(pos,N,L,n_events,epsilon,I);
diffs=diff(xsamples,1,2);
E=0.5*sum(diffs.^2,2);
burn=floor(0.1*n_events);
E = E(burn+1:end);
Emean_theoretical=(L^2/N+N-1)/2;
Emean_sample=mean(E);
histogram(E,"Normalization","pdf");
xline(Emean_sample,'g','LineWidth',2);
xline(Emean_theoretical,'--r','LineWidth',2);
xlabel("Energy");
ylabel("Probability Density");
title('Energy Distribution of HMC Samples');
legend('Sample Distribution','Sample Mean','Theoretical Mean');
```



We can see from the plots that all three algorithms compute the correct b-independent mean energy giving sufficient sample sizes and reasonably defined parameters. Compared to the mertoplis algorithm instead of taking a random walk we assign each particle a momentum, reducing the rejection rate. Through HMC we construct a joint probability distribution that combines the potential and the knetic energy hence we must use $E=U+K$.